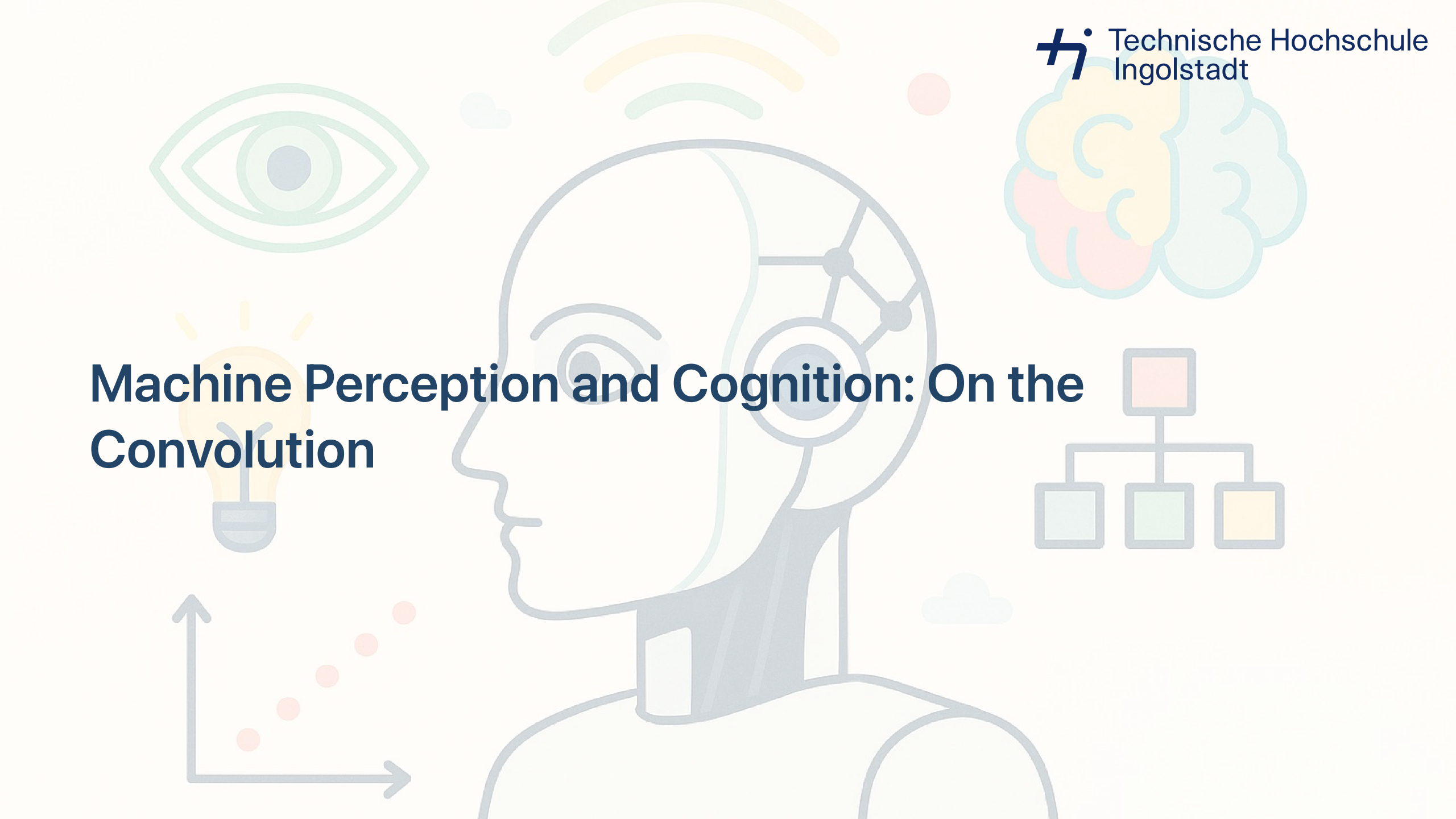
Technische Hochschule Ingolstadt



Machine Perception and Cognition: On the Convolution

Convolution

From Wikipedia, page "Convolution":

> In mathematics (in particular, functional analysis), **convolution** is a mathematical operation on two functions f and g that produces a third function $f * g$, as the integral of the product of the two functions after one is reflected about the y-axis and shifted. The term *convolution* refers to both the resulting function and to the process of computing it. The integral is evaluated for all values of shift, producing the convolution function. The choice of which function is reflected and shifted before the integral does not change the integral result (see commutativity). Graphically, it expresses how the 'shape' of one function is modified by the other.

Convolution

From Wikipedia, page "Convolution":

> Convolution has applications that include probability, statistics, acoustics, spectroscopy, signal processing and image processing, computer vision and human vision, geophysics, engineering, physics, and differential equations.

Convolution

Continuous signals:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t - \tau)\, d\tau$$

Discrete signals:

$$(f * g)(n) = \sum_{k=-\infty}^{\infty} f(k)\, g(n - k)$$

Cross-correlation

From Wikipedia, page "Cross-correlation":

> In signal processing, **cross-correlation** is a measure of similarity of two series as a function of the displacement of one relative to the other. This is also known as a sliding dot product or sliding inner-product. It is commonly used for searching a long signal for a shorter, known feature. It has applications in pattern recognition, single particle analysis, electron tomography, averaging, cryptanalysis, and neurophysiology. The cross-correlation is similar in nature to the convolution of two functions.

Cross-correlation

Continuous signals:

$$(f \star g)(t) \triangleq \int_{-\infty}^{\infty} f(\tau)\, g(t+\tau)\, d\tau$$

Discrete signals:

$$(f \star g)(n) = \sum_{k=-\infty}^{\infty} f(k)\, g(n+k)$$

Why convolutions in computer vision?

Convolutions are

- commutative: $f * g = g * f$
- associative: $f * (g * h) = (f * g) * h$

Convolution: How does one function change the shape of another?

Cross-correlation: How similar are two functions?

Discrete cross-correlation in 2D

For an image I of size $H \times W$ and a kernel K of size $k \times k$:

$$(I \star K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I[i+m,\ j+n] \cdot K[m, n]$$

The kernel **slides** across the image. At each position (i, j), it computes the **dot product** between the overlapping image patch and the kernel.

Output size (no padding, stride 1):

$$H_{\text{out}} = H - k + 1, \qquad W_{\text{out}} = W - k + 1$$

Each output pixel aggregates information from a **local neighbourhood** — this is called the **receptive field** of that output.

Discrete convolution in 2D

Animation of a convolution in computer vision. Anyone notice anything?

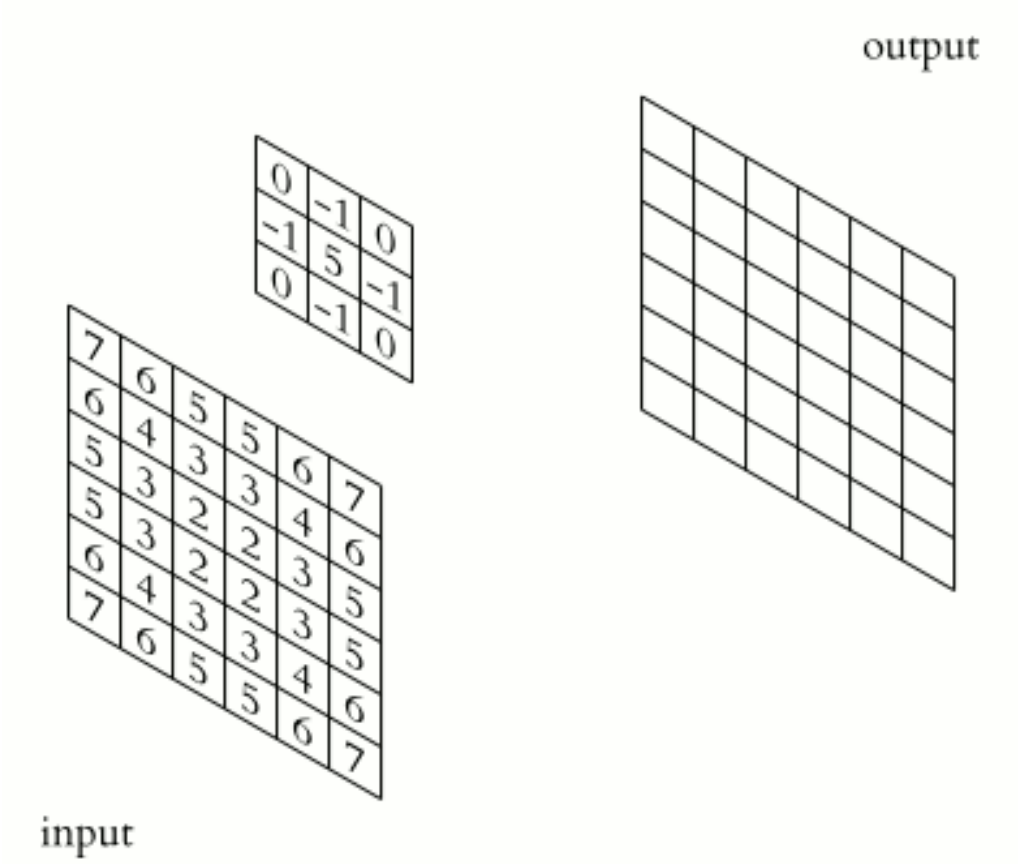
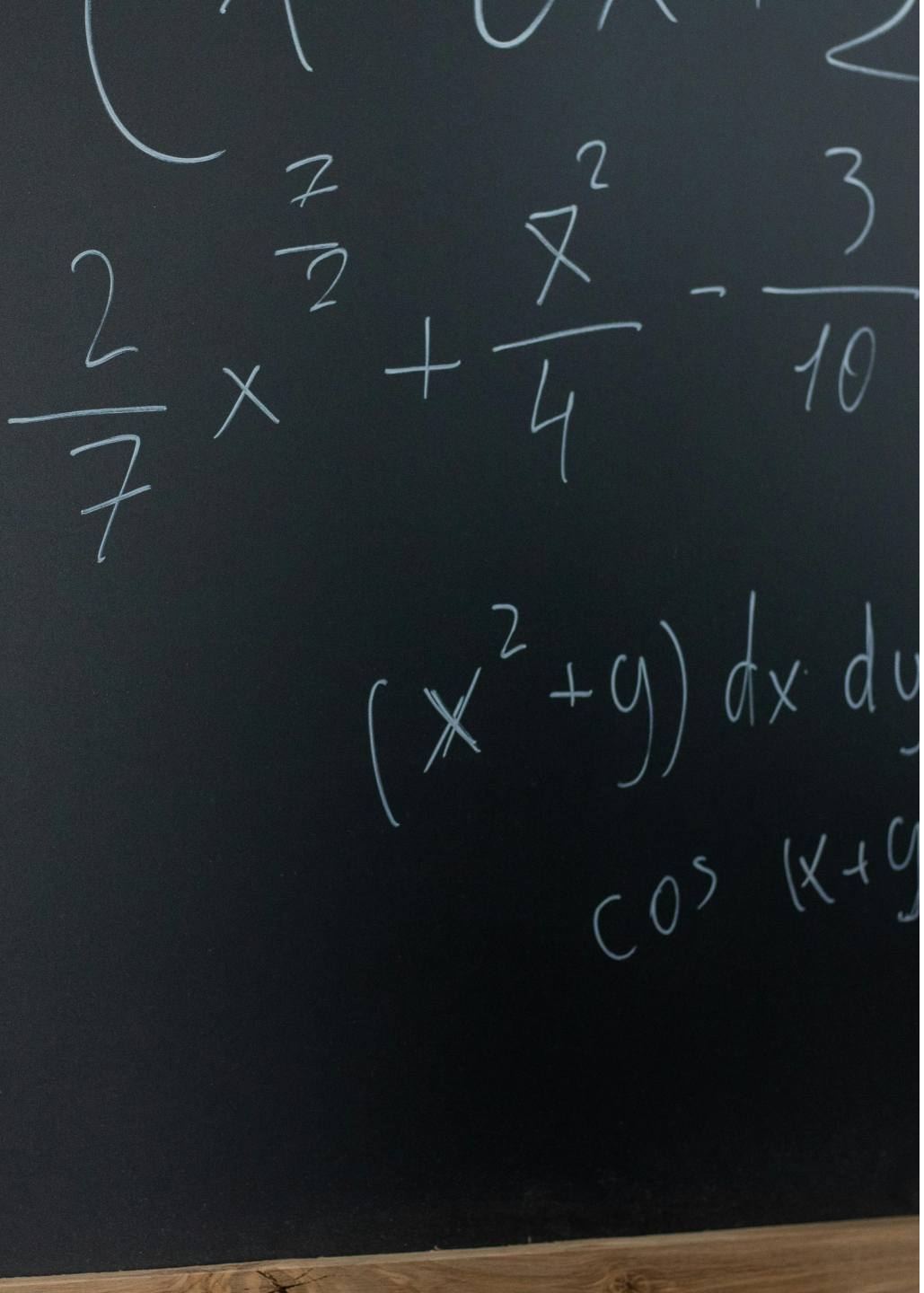


Image source: Michael Plotke, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://en.wikipedia.org/wiki/File:2D_Convolution_Animation.gif



Discrete convolutions in 2D

That's not a convolution! That's a cross-correlation! Does it matter? Not really, since we will learn the coefficients of the convolution anyway and can "learn them mirrored" if that is what solves the problem...

Image source: https://www.pexels.com/photo/mathematical-equation-written-on-blackboard-6238050/

Padding

Applying a $k \times k$ kernel shrinks the output by $k - 1$ pixels per dimension. **Padding** adds p rows/columns of zeros around the image before the kernel is applied:

$$H_{\text{out}} = H - k + 1 + 2p$$

Stride

The **stride** s sets how many pixels the kernel moves at each step (default: $s = 1$).

Output size with padding p and stride s:

$$H_{\text{out}} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1$$



What can we do with convolutions?

Image source: https://www.pexels.com/photo/question-marks-on-paper-crafts-5428831/

Gaussian blur

A **Gaussian kernel** smooths an image by averaging each pixel with its neighbours, weighted by a 2D Gaussian:

$$G_\sigma[i, j] = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right)$$

Normalised 3×3 approximation ($\sigma \approx 0.85$):

$$K_G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

First-order derivative

What happens in this discrete cross-correlation?

$$(f \star k)[i] = f[i] \cdot (-1) + f[i+1] \cdot (+1) = f[i+1] - f[i]$$

with $k = [-1, +1]$ and f being our discrete signal.

Sobel operator

$$k_H = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad k_V = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix}$$

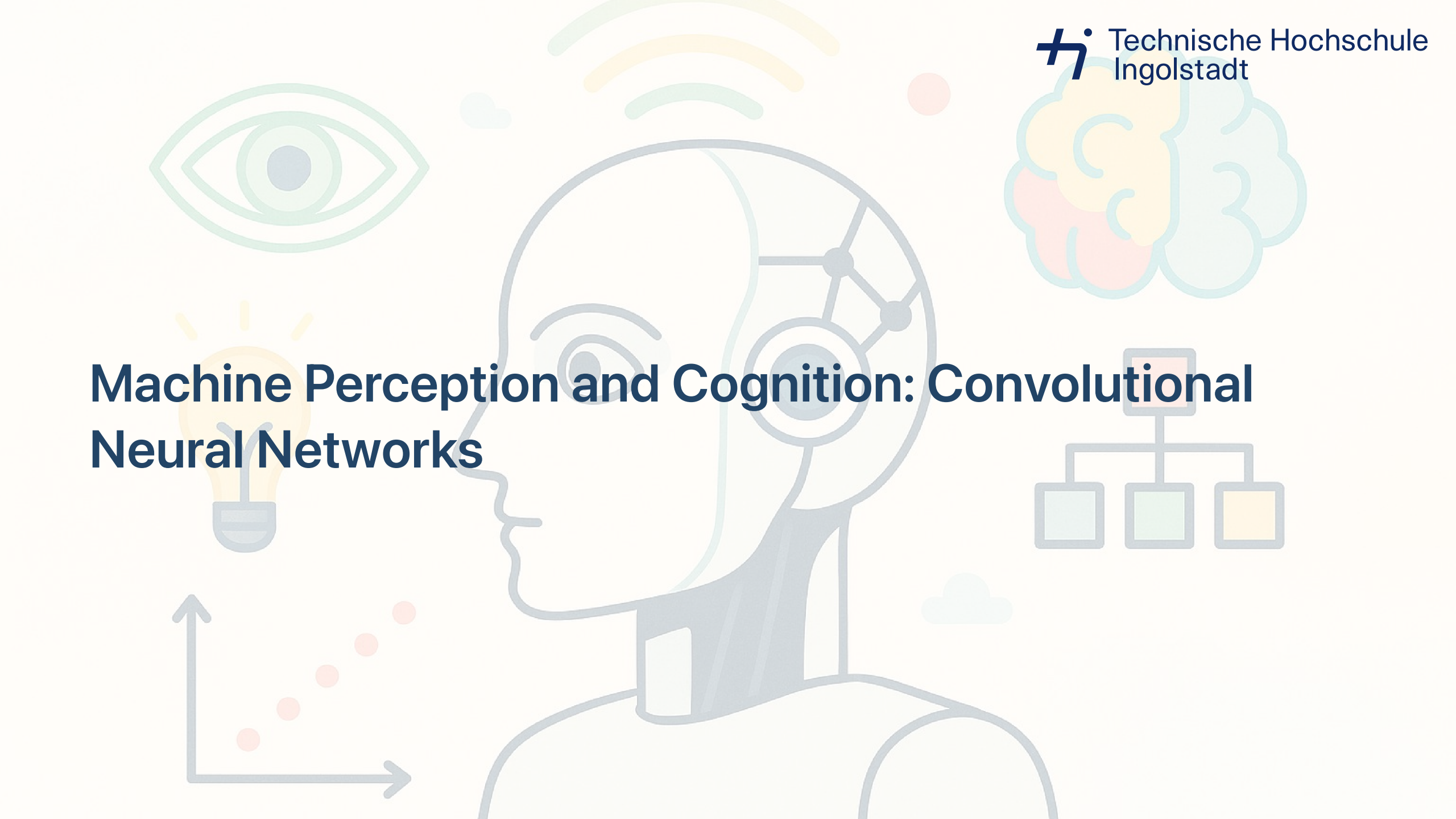
Edge-detection using the first-order derivative along horizontal and vertical lines.

On the Convolution

Recap!

- 📶 Convolutions and cross-correlation allow us to find correlation between signals.
- 🔎 They can be used to find patterns in discrete data, e.g. images.
- 📐 The output size is controlled by the **kernel size**, **padding** (p), and **stride** (s): $H_{\text{out}} = \lfloor (H - k + 2p)/s \rfloor + 1$.
- 🌫️ The **Gaussian kernel** smooths images, the **Sobel** kernel detect edges.
- ⌜ Convolutions can thus be used to detect e.g. edges in images.
- 🧠 Convolutions in computer vision are most often parameterized by learning algorithms.

Technische Hochschule Ingolstadt



Machine Perception and Cognition: Convolutional Neural Networks

From MLP to CNN — the challenge of image data

Consider a small image of 224×224 pixels in RGB (3 channels):

$$224 \times 224 \times 3 = 150528 \text{ input values}$$

For a single fully connected hidden layer with 1000 neurons:

$$150528 \times 1000 = 150.5 \text{ million parameters} — \text{just for the first layer!}$$

This does not scale:

- 📦 Huge memory requirements
- 📉 Prone to severe overfitting
- 🐌 Computationally expensive

The problem with flattening images

Flattening an image into a vector **destroys all spatial structure**:

- 🔲 Nearby pixels belong together (edges, corners, textures)
- ↔️ A translation of an object by one pixel → a completely different input vector
- 🤦 An MLP must learn to recognize each pattern separately at **every possible position**
- 📐 Spatial relationships between pixels are not encoded in a flat vector

🐱 A cat in the top-left corner and a cat in the bottom-right corner look **completely different** to an MLP!

Key ideas behind CNNs

CNNs address these problems through three structural constraints:

1. 🔍 **Local connectivity**: each neuron connects only to a small local region of the input
2. 🔁 **Weight sharing**: the same kernel is applied at every spatial position
3. 🏗️ **Spatial hierarchy**: stacking layers builds increasingly complex features

Together, these constraints dramatically reduce the number of parameters and build in spatial awareness.

Local connectivity

In a **fully connected** layer: every output neuron connects to **all** inputs.

In a **convolutional** layer: every output neuron connects only to a **small local region**:

- The size of this region is the **receptive field** (e.g., 3×3 pixels)
- For a kernel of size $k \times k$ (e.g., $k = 3$): each output neuron depends on only $k^2 = 9$ input values
- Local patterns such as edges or corners are detectable from small regions

➡️ Local connectivity is both **computationally efficient** and **spatially meaningful**.

Weight sharing

In a **fully connected** layer: every connection has its own weight.

In a **convolutional** layer: the **same kernel** is applied at every spatial position:

- Assumption: **the same pattern can occur anywhere in the image**
- A 3×3 kernel on a 224×224 input: 9 weights shared across $\approx 50{,}000$ positions
- This is called **translational invariance**: shifting the input shifts the feature map

➡️ Weight sharing reduces parameters by orders of magnitude and improves generalization.

A 1D convolutional neural network

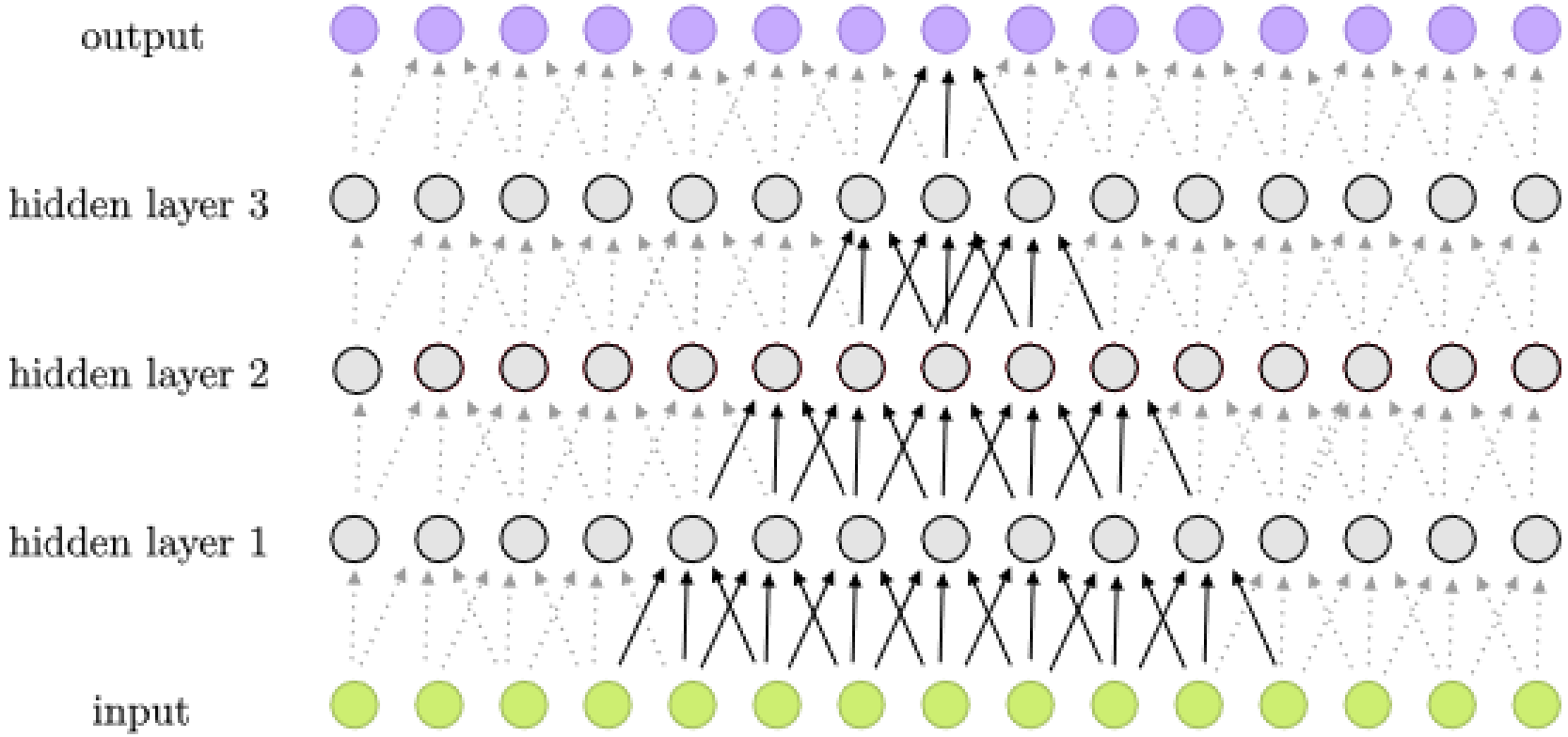


Image source: Vicente Oyanedel M., CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:1D_Convolutional_Neural_Network_feed_forward_example.png

Receptive field in a 2D convolutional neural network

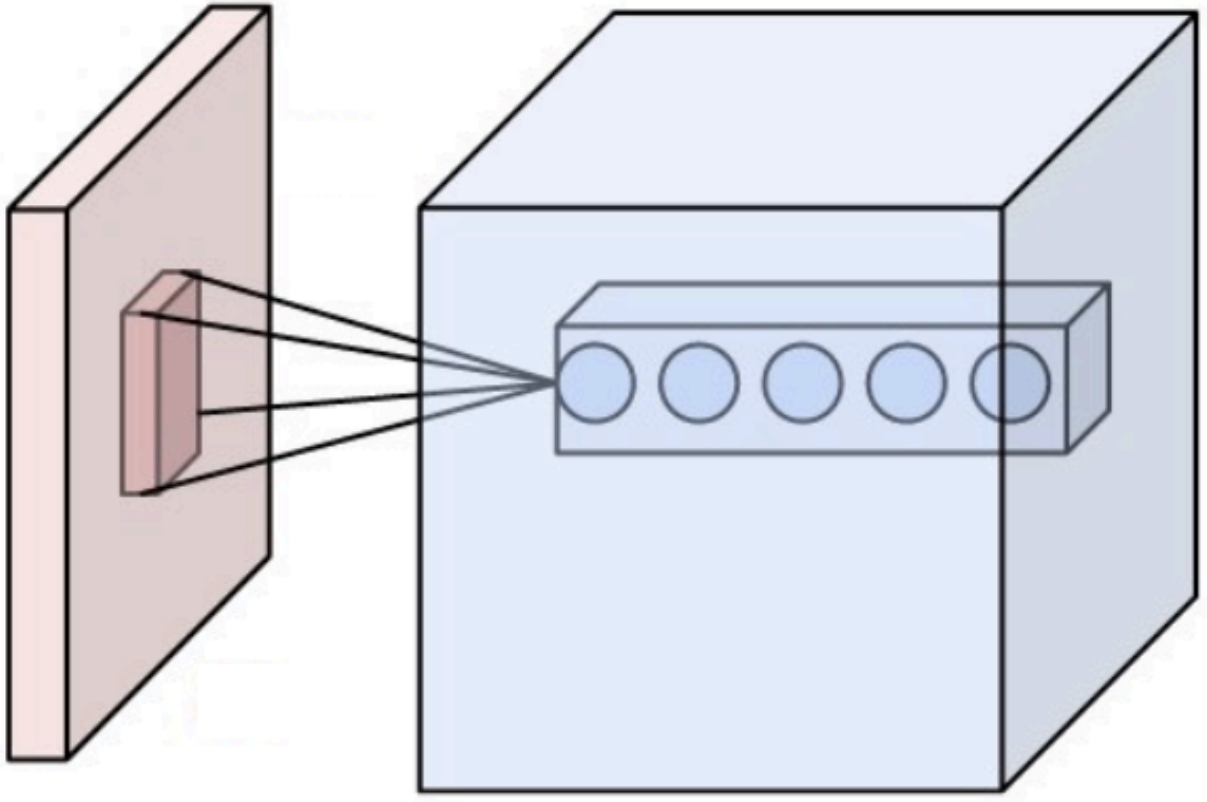
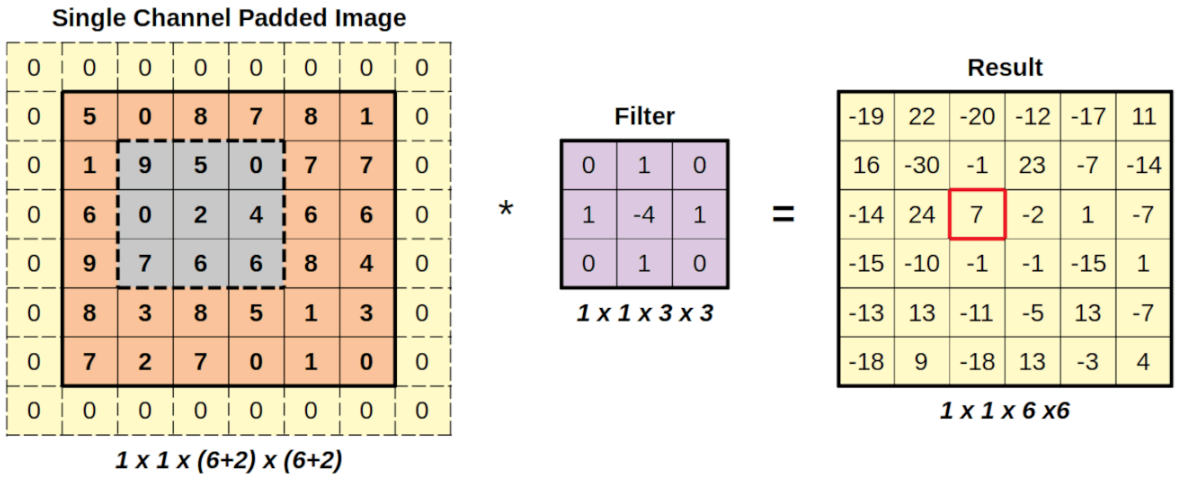


Image source: Aphex34, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Conv_layer.png

Working example for a 2D convolution



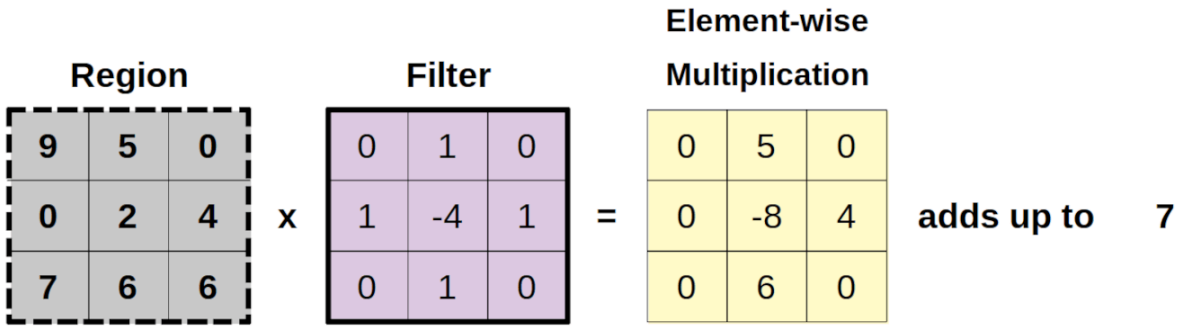


Image source: Daniel Voigt Godoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Convolutional_neural_network,_convolution_worked_example.png

Convolutional layer — matrix-vector form

For a 1D convolutional layer with kernel $\mathbf{w} \in \mathbb{R}^k$, bias b, and stride s, the output at position i is:

$$z_i = \sum_m w_m \cdot x_{i \cdot s + m} + b = \mathbf{w}^\top \mathbf{x}_i + b$$

where $\mathbf{x}_i = \begin{bmatrix} x_{i \cdot s} & \cdots & x_{i \cdot s + k - 1} \end{bmatrix}^\top$ is the local input patch at position i.

Stacking all output positions $i = 0, \ldots, m-1$ into a vector $\mathbf{z} \in \mathbb{R}^m$ gives the **matrix-vector form**:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

with weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ (one kernel row per output position) and bias $\mathbf{b} = b \cdot \mathbf{1} \in \mathbb{R}^m$.

Convolutional layer — augmented form

Folding the bias into an augmented weight matrix $\mathbf{W} \in \mathbb{R}^{m \times (n+1)}$ (bias in last column) and an augmented input $\mathbf{x} \in \mathbb{R}^{n+1}$ (last entry $= 1$):

$$\mathbf{x} = \begin{bmatrix} x_1 & \cdots & x_n & 1 \end{bmatrix}^\top \in \mathbb{R}^{n+1}$$

The complete forward pass of a convolutional layer:

$$\mathbf{z} = \mathbf{W}\mathbf{x}, \qquad \mathbf{a} = \sigma(\mathbf{z})$$

- $\mathbf{x} \in \mathbb{R}^{n+1}$: augmented input (last entry $= 1$)
- $\mathbf{W} \in \mathbb{R}^{m \times (n+1)}$: weight matrix with bias in last column
- $\mathbf{z} \in \mathbb{R}^m$: pre-activation vector, $\mathbf{a} \in \mathbb{R}^m$: feature map activations
- The structure is **identical to an MLP layer** — weight sharing is implicit in the shared kernel parameters $\mathbf{w}$

Subsampling

Subsampling in a convolutional neural network is the process of reducing the spatial resolution of a feature map, often by applying a **pooling** operation. **Max pooling** propagates the maximum activation within its receptive field, while **average pooling** propagates the arithmetic mean of the activations within its receptive field.

Max pooling

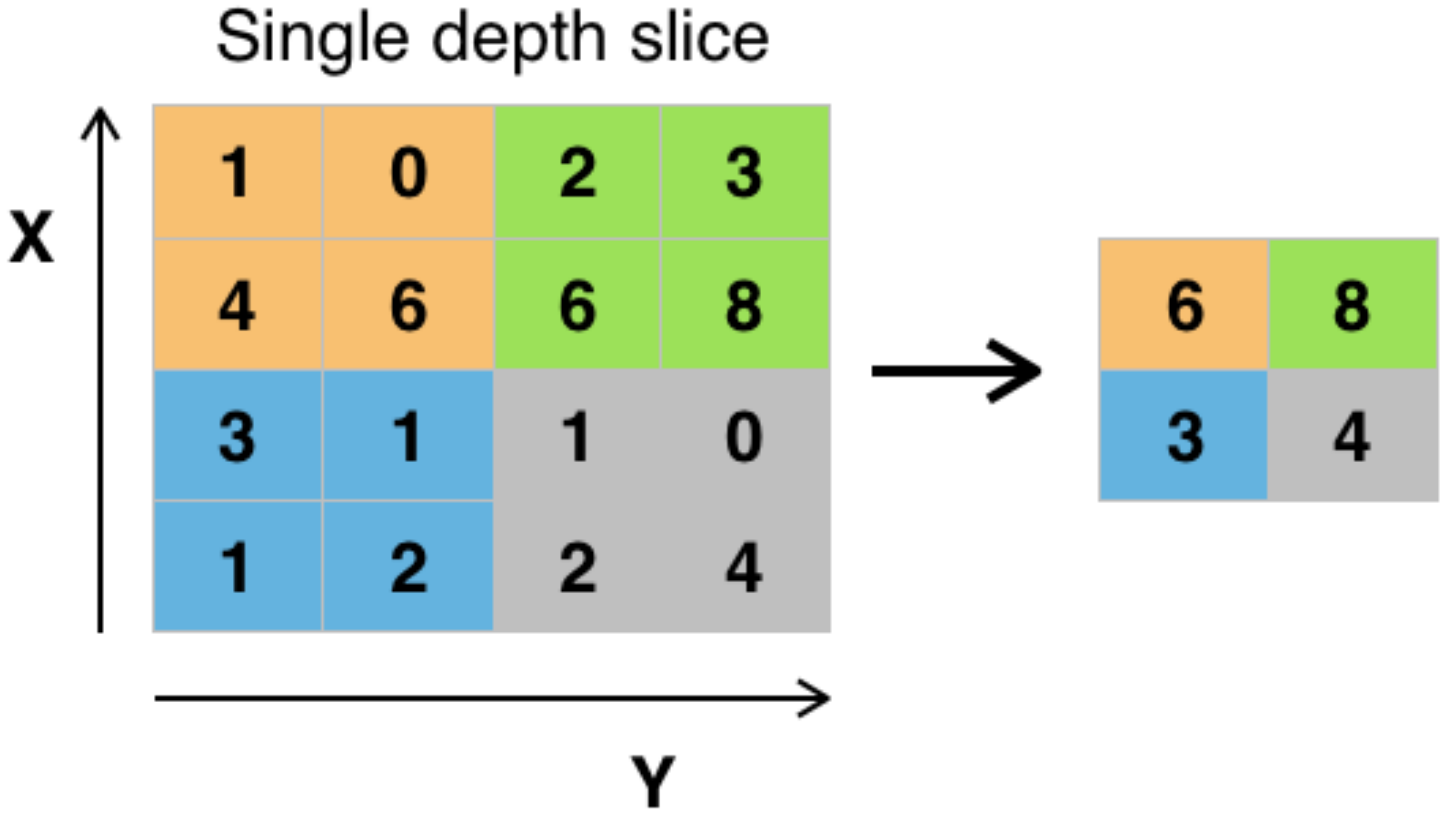


Image source: Aphex34, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://en.wikipedia.org/wiki/File:Max_pooling.png

Max pooling

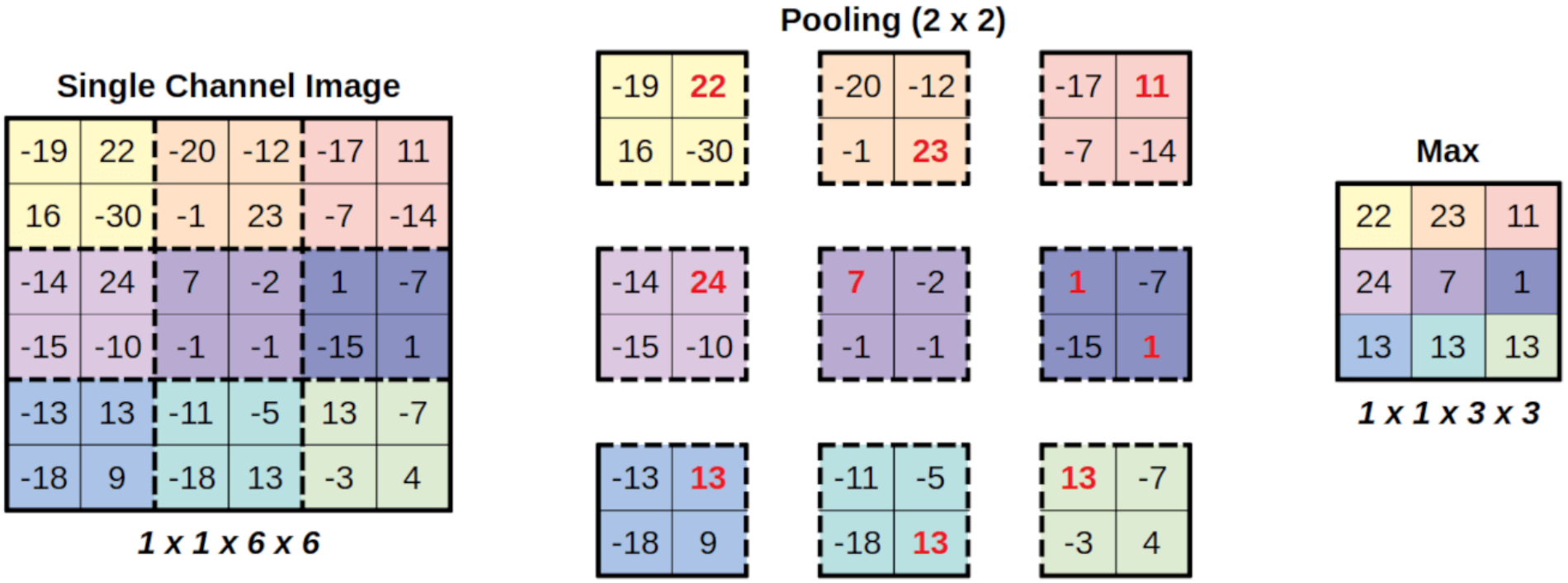


Image source: Daniel Voigt Godoy, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://en.wikipedia.org/wiki/File:Convolutional_neural_network,_maxpooling.png

Max pooling

Max pooling propagates the **maximum activation** within each pooling window:

$$y_{i,j} = \max_{m}^{k} \max_{n}^{k} x_{i \cdot s+m,\ j \cdot s+n}$$

with x being the pooling input, y the output and s the step size for pooling.

- Retains the strongest activation in each local region
- More commonly used than average pooling in modern architectures
- Applied **channel-wise**: each feature map is pooled independently

Average pooling

Average pooling computes the **arithmetic mean** of activations in each pooling window:

$$y_{i,j} = \frac{1}{k^2} \sum_{m}^{k} \sum_{n}^{k} x_{i \cdot s+m,\ j \cdot s+n}$$

with x being the pooling input, y the output and s the step size for pooling.

- Provides a smoother summary of the feature map compared to max pooling
- Less commonly used in modern architectures than max pooling
- Applied **channel-wise**: each feature map is pooled independently

Pooling and spatial invariance

Pooling contributes to **spatial invariance**: small translations of the input lead to the same pooling output:

- If a feature appears at position (i, j) or $(i + 1, j)$, max pooling returns the same maximum value
- This is **local invariance** — not full translation invariance
- Full translation invariance is achieved through the combination of: convolutional weight sharing + pooling + deep hierarchy

CNNs are translation-invariant at pooling layers, not at convolutional layers!

Classic CNN architecture

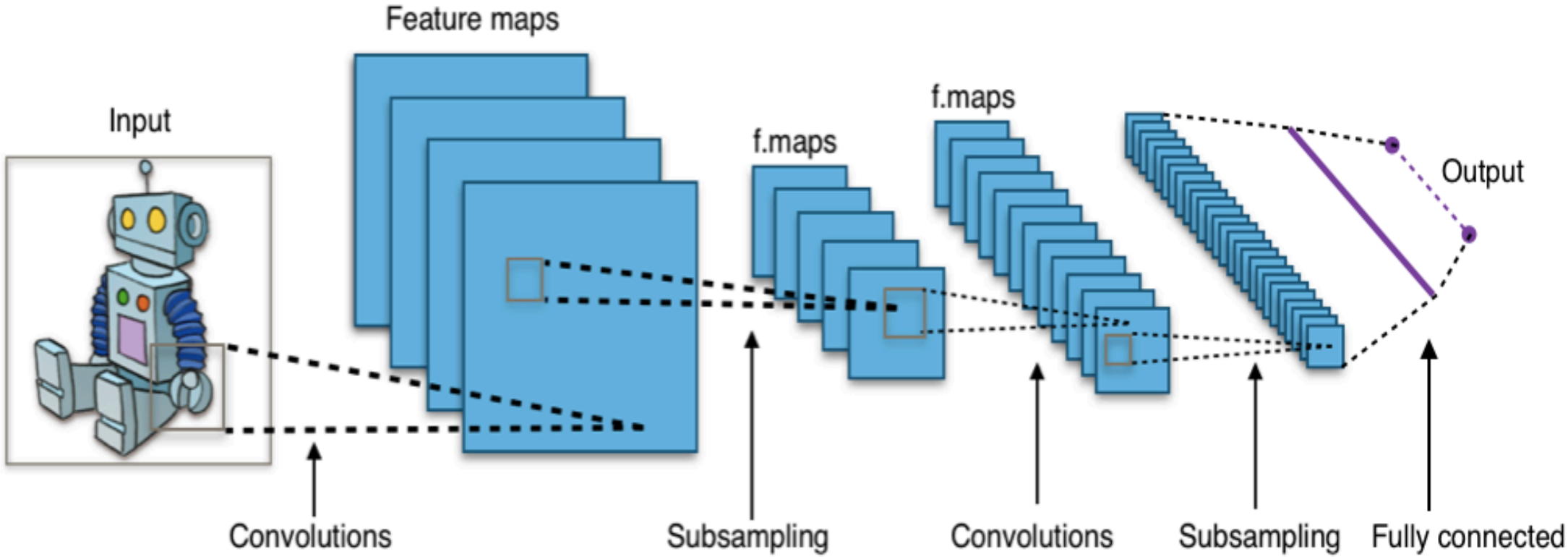


Image source: Aphex34, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Typical_cnn.png

Feature maps are organized in 3D tensors

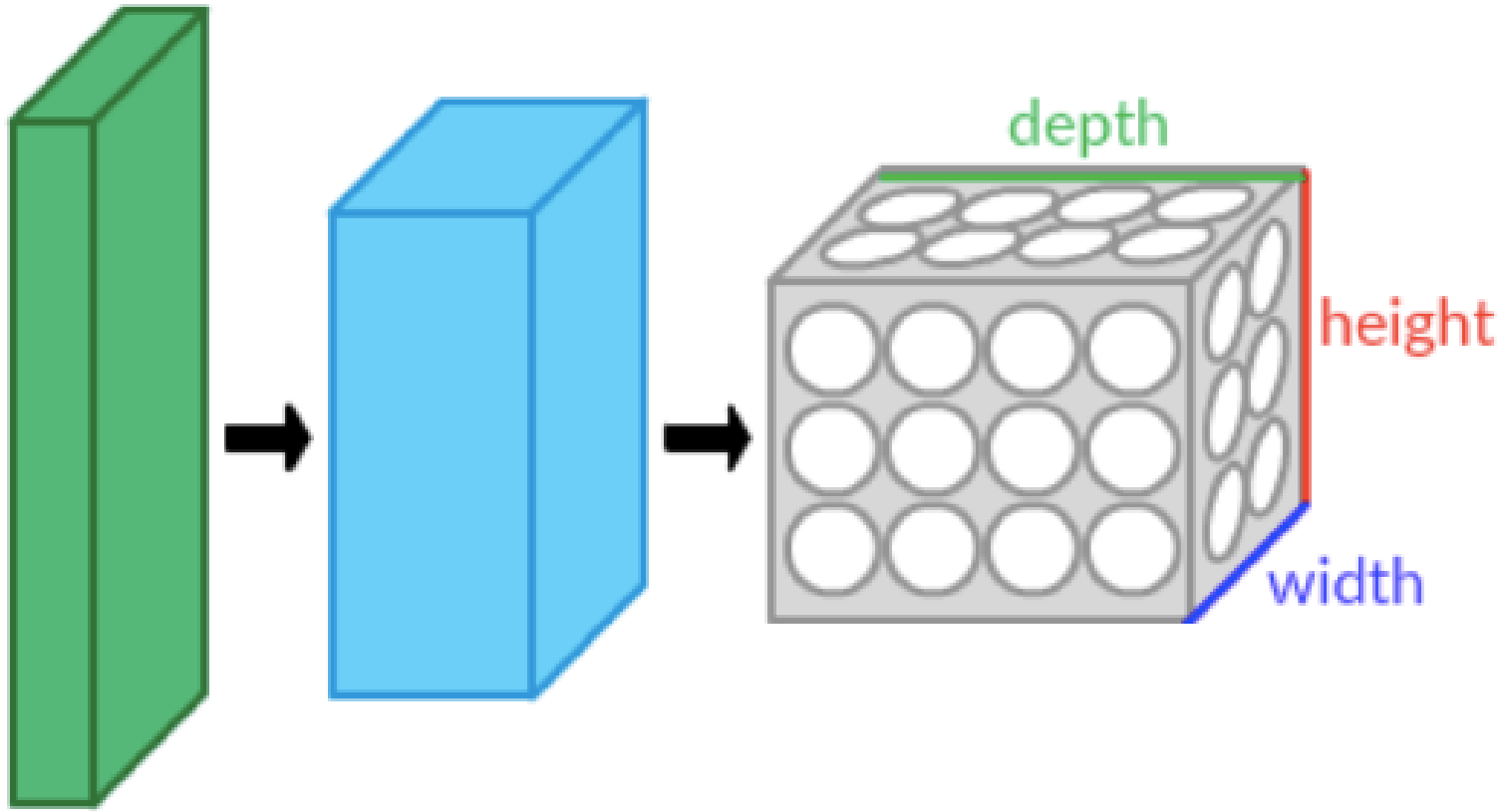
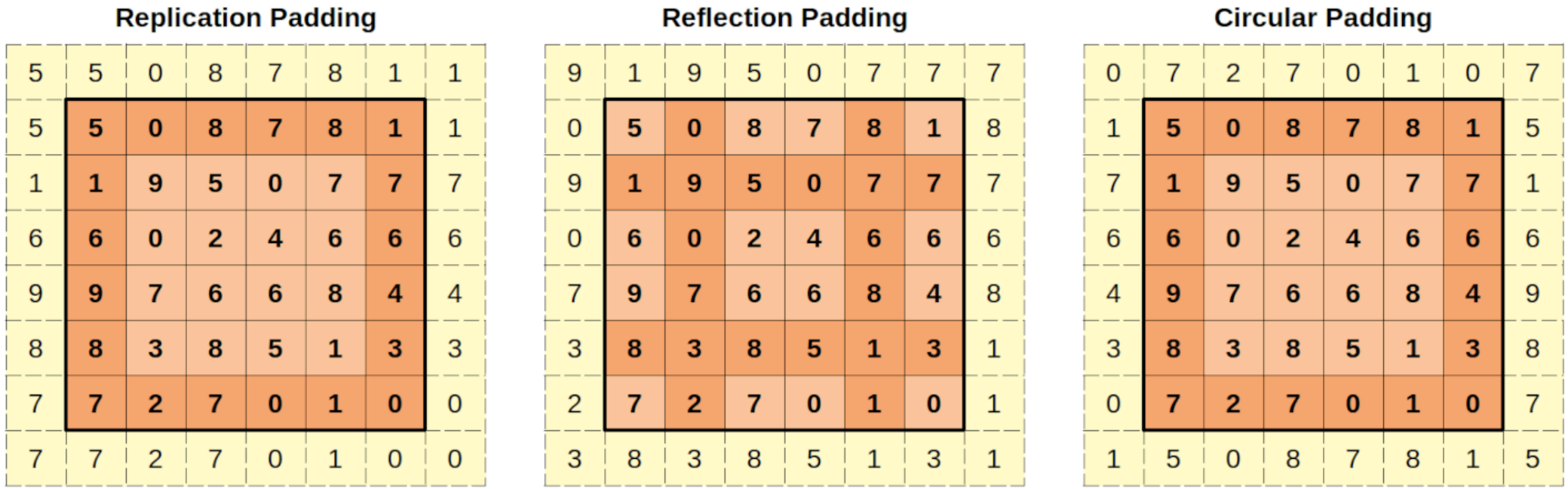


Image source: Aphex34, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Conv_layers.png

Different padding strategies



Zero padding is commonly used for simplicity reasons, but strictly speaking breaks patterns in the feature map.

Image source: Daniel Voigt Godoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://en.wikipedia.org/wiki/File:Convolutional_neural_network,_boundary_conditions.png

Learned kernels

Unlike hand-designed filters (Sobel, Gaussian), CNN kernels are **learned from data**:

- During training, kernel weights are optimized to minimize a task-specific loss
- Different kernels automatically learn to detect different patterns
- The same training process applies to any visual task: classification, detection, segmentation

Key insight: Instead of manually specifying what features to extract, we let the data determine which features are most useful.

Multi-channel input: RGB images

A color image has $C_{\text{in}} = 3$ channels (R, G, B). Each filter must cover **all input channels**:

- A 3×3 filter for a 3-channel input has size $3 \times 3 \times 3 = 27$ values
- The output of the filter at position (i, j) is:

$$z_{i,j} = \sum_{c}^{C_{\text{in}}} \sum_{m}^{k} \sum_{n}^{k} W_{m,n,c} \cdot x_{i \cdot s+m,\ j \cdot s+n,\ c} + b$$

with m and m being relative coordinates within the kernel size k.

- The filter simultaneously looks at patterns across **all input channels**

Multiple filters → multiple feature maps

A convolutional layer applies C_{out} different filters:

- Each filter $f = 1, \ldots, C_{\text{out}}$ produces one **feature map** of size $H_{\text{out}} \times W_{\text{out}}$
- The C_{out} feature maps are stacked into a 3D output volume: $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$
- Each filter detects a **different pattern** in the input

Total number of parameters in a convolutional layer:

$$(k \times k \times C_{\text{in}} + 1) \times C_{\text{out}}$$

(+1 for the bias term per filter)

Bias in convolutional layers

Each filter has **one scalar bias** b_f:

$$z_{i,j,f} = \sum_{c}^{C_{\text{in}}} \sum_{m}^{k} \sum_{n}^{k} W_{m,n,c,f} \cdot x_{i \cdot s+m,\ j \cdot s+n,\ c} + b_f$$

- The bias shifts the activation threshold of the filter
- The same bias is added at **every spatial position** (shared like the kernel weights)

Output dimensions — example

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2p}{s} \right\rfloor + 1$$

Given:

- Input: $32 \times 32 \times 3$ (RGB image)
- Kernel size: 3×3, padding $p = 1$, stride $s = 1$, $C_{\text{out}} = 64$ filters

$$H_{\text{out}} = \lfloor (32 - 3 + 2 \cdot 1)/1 \rfloor + 1 = 32$$

Output volume: $32 \times 32 \times 64$ ✓

For $p = 0$, $s = 2$:

$$H_{\text{out}} = \lfloor (32 - 3 + 0)/2 \rfloor + 1 = 15$$

Output volume: $15 \times 15 \times 64$ ✓

Effective receptive field

How large a region of the input does a neuron in layer l "see"?

For l layers of $k \times k$ convolutions with stride 1 and no pooling:

$$r_l = 1 + l \cdot (k - 1)$$

Layers	$k = 3$	$k = 5$
1	3×3	5×5
2	5×5	9×9
3	7×7	13×13
5	11×11	21×21

➡️ Stacking small kernels achieves large receptive fields efficiently.

Non-linear activation in CNNs

Convolution is a **linear** operation. Without non-linearity, stacking convolutional layers is equivalent to one big convolution.

After each convolutional layer, apply a **non-linear activation function** element-wise:

$$\mathbf{a} = \sigma(\mathbf{z})$$

- Most commonly used: **ReLU**: $\sigma(z) = \max(0, z)$
- The same non-linearities used in MLPs — applied to every element of the feature map

Batch normalization

Batch normalization (Ioffe & Szegedy, 2015) normalizes activations across a mini-batch of size m:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad \mu_B = \frac{1}{m}\sum_{i=1}^{m} z_i, \quad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(z_i - \mu_B)^2$$

Batch normalization

Why do batch normalization?

- ⚡ Stabilizes and **accelerates training**
- 🎯 Reduces sensitivity to weight **initialization**
- 🛡️ Acts as mild **regularization**
- 📍 Typically applied **between** convolution and activation function

Dropout

Dropout (Srivastava et al., 2014) randomly sets activations to zero during training:

$$a_i' = \begin{cases} 0 & \text{with probability } p \\ \dfrac{a_i}{1-p} & \text{with probability } 1-p \end{cases}$$

Dropout

Why do dropout?

- During **inference**, no dropout is applied (all neurons active)
- The rescaling by $\frac{1}{1-p}$ ensures the expected output is unchanged
- Prevents co-adaptation: no neuron can rely on specific other neurons being present
- Equivalent to training an ensemble of 2^n subnetworks simultaneously
- Typical values: $p = 0.5$ for fully connected layers, $p = 0.1$–0.2 for convolutional layers

The standard CNN building block

Modern CNNs are built from repeated **building blocks**:

$$\text{Input} \xrightarrow{\text{Conv.}} \xrightarrow{\text{BN}} \xrightarrow{\text{ReLU}} \left[\xrightarrow{\text{Pool}}\right] \text{Output}$$

- **Convolution**: applies learned filters, extracts local spatial features
- **BN**: normalizes activations, stabilizes and accelerates training
- **ReLU**: introduces non-linearity
- **Pool** (optional): reduces spatial dimensions, increases invariance

This block is the **fundamental unit** of almost all modern CNN architectures.

Stacking CNN blocks

Stacking multiple blocks creates a **deep hierarchy**:

- 🔽 Spatial resolution **decreases**
- 🔼 Number of channels **increases**
- 📈 Deeper networks can represent **more complex functions**
- 🧠 Increasing **abstraction** in deeper layers

1×1 convolution

A 1×1 convolution mixes information **across channels** without any spatial aggregation:

$$z_{i,j,f} = \sum_{c=1}^{C_{\text{in}}} W_{c,f} \cdot x_{i,j,c} + b_f$$

Each spatial position (i, j) is processed independently — only the channel dimension is mixed.

Key uses:

- **Channel reduction**: project C_{in} to a smaller C_{out} before expensive 3×3 convolutions
- **Channel expansion**: increase depth without changing spatial resolution
- **Nonlinear projection**: when followed by an activation function

Global average pooling

Global average pooling (GAP) collapses the entire spatial extent of each feature map to a single value:

$$y_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_{i,j,c}$$

Reduces a feature volume $H \times W \times C$ to a vector of length C.

Advantages over flattening + fully connected layers:

- 🪶 Far **fewer parameters** — no FC weights needed for the spatial part
- 🔄 More spatially **invariant** — the output does not depend on feature location
- 📐 Handles **variable input sizes** without architectural changes

Parameters in a convolutional layer

For a convolutional layer with:

- Kernel size: $k \times k$
- Input channels: C_{in}
- Output channels (filters): C_{out}

$$\text{Parameters} = (k^2 \cdot C_{\text{in}} + 1) \times C_{\text{out}}$$

Example: $k = 3, C_{\text{in}} = 64, C_{\text{out}} = 128$:

$$(9 \times 64 + 1) \times 128 = 577 \times 128 = 73{,}856 \text{ parameters}$$

The parameter count is **independent of the input spatial size** $H \times W$!

MLP vs. CNN — parameter comparison

Compare processing a $32 \times 32 \times 3$ image (CIFAR-10 size):

Layer type	Config	Parameters
FC (MLP)	$3072 \to 1024$	$3{,}146{,}752$
Conv.	$3 \times 3, C_{\text{in}} = 3, C_{\text{out}} = 64$	$1{,}792$
Conv.	$3 \times 3, C_{\text{in}} = 64, C_{\text{out}} = 128$	$73{,}856$

🔑 Even a **single** FC layer mapping a 32×32 image to 1024 neurons needs **3.1 million parameters**. By contrast, a convolutional layer achieves richer spatial feature extraction with orders of magnitude fewer parameters.

Weight sharing — the key efficiency gain

Without weight sharing (hypothetical "locally connected" layer):

$$\text{Parameters} = k^2 \cdot C_{\text{in}} \cdot H_{\text{out}} \cdot W_{\text{out}} \cdot C_{\text{out}}$$

With weight sharing (standard convolutional layer):

$$\text{Parameters} = k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$$

Reduction factor: $H_{\text{out}} \cdot W_{\text{out}}$

For a 224×224 feature map: $224 \times 224 = 50{,}176\times$ fewer parameters!

➡️ Weight sharing makes deep CNNs tractable.

What do CNNs learn?

CNNs learn a **hierarchical representation** of visual patterns:

- 🎨 **Early layers**: detect low-level features — oriented edges, color gradients, blobs
- 🧩 **Middle layers**: combine low-level features into textures, patterns, and object parts
- 🏷️ **Late layers**: encode high-level semantic concepts — faces, wheels, text, animals

The hierarchy is **automatically discovered** from training data:

- No manual feature engineering
- Features learned for one task (e.g., object classification) often transfer to other tasks

First layer filters: edges and orientations

The filters learned in the first convolutional layer of a CNN trained on natural images resemble:

- ↗️ Oriented edge detectors
- 🌈 Color gradients
- 🔄 Various spatial frequencies

This is **not a coincidence** — these are the most informative low-level patterns in natural images.

Higher layer features

As layers deepen, feature complexity increases:

- **e.g. Layer 2–3**: Textures (grids, stripes, dots, fur, bark, ...)
- **e.g. Layer 4–5**: Object parts (wheels, eyes, leaves, beaks, ...)
- **Final convolutional layers**: Whole objects and scene-level features

Pros of CNNs

- ✅ **Weight sharing**: orders of magnitude fewer parameters than fully connected layers
- ✅ **Local connectivity**: spatially meaningful, efficient feature extraction
- ✅ **Translational invariance**: feature detection independent of spatial location
- ✅ **Hierarchical features**: automatically learned at multiple scales
- ✅ **Strong empirical performance**: state-of-the-art on many visual tasks
- ✅ **Transfer learning**: large pre-trained models available for many domains

Cons of CNNs

- ❌ **Not rotation/scale invariant**: separate learning required for rotated/scaled variants
- ❌ **Data hungry**: need large labeled datasets for training from scratch
- ❌ **Computationally expensive**: large models require GPUs and significant energy
- ❌ **Black box**: hard to interpret what features are being used for a decision
- ❌ **Adversarial vulnerability**: small imperceptible perturbations can fool the network
- ❌ **Fixed receptive field**: may miss long-range dependencies

Convolutional Neural Networks

Recap!

- 🌄 **Motivation**: MLP is impractical for images — too many parameters, ignores spatial structure.
- 🔍 **Local connectivity** and 🔁 **weight sharing** are the two key structural constraints.
- 📐 A convolutional layer maps an $H \times W \times C_{\text{in}}$ volume to $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$.
- 🏊 **Pooling** reduces spatial resolution and increases spatial invariance.
- 📊 **Batch normalization** stabilizes training; **dropout** prevents overfitting.
- 🏗️ **Stacking blocks** (Conv. → BN → ReLU → Pool) builds hierarchical and abstract features.
- 🔢 Parameters per convolutional layer: $(k^2 \cdot C_{\text{in}} + 1) \times C_{\text{out}}$.